Better lookups for `map` and `unordered_map`

Document #:	P3091R0
Date:	2024-02-06 10:36 EST
Project:	Programming Language C++
Audience:	LEWGI

Reply-to:	Pablo Halpern <phalpern@halpernwightsoftware.com>

Contents

§ 1 Abstract

The most convenient way to look up an element of a `map` or `unordered_map` is to use the index operator, i.e., `theMap[key]`. This operator has a number of limitations, however: 1) it does not work for `const` containers, 2) it does not work if the mapped type is not default-constructible, 3) it modifies the container if the key is not found 4) a default-constructed value is not always the desired value when the key is not found. These limitations often require that the user resort to using the `find` member function, which returns an iterator that points to a `pair` and typically leads to more complex code having at least one `if` statement and/or duplicate lookup operations. This paper take inspiration from other languages, especially Python, and proposes three simple member functions (`get`, `get_ref`, and `get_as`) that simplify lookups for `map` and `unordered_map` in situations where the index operator is suboptimal.

§ 2 Motivation

Just about every modern computer language has, as part of the language or its standard library, one or more associative containers that uniquely associates a key with a mapped value, variously called `map`, `hash_map`, `dictionary`, or something similar. In C++ we have `std::map`, `std::unordered_map`, and eventually (per [P0429R9] and [P2767R2]) `std::flat_map`. The easy-to-write and easy-to-read expression for getting a value for an associated key is simply, `theMap[key]`; in other words, a mapped value is retrieved (and often set) using the index operator, which returns a reference to the value associated with the key. Unfortunately, the index operator in the C++ associative containers has a number of shortcomings compared to many other languages:

— It works only for non-`const` containers.

— It works only for default-constructible mapped types

— The default-constructed object (when available) is not always the correct identity for a given use case.

— It modifies the container when the key is absent.

Consider a `std::map` named `theMap`, that maps an integer key to a floating-point value, modeling a sparse array of `double`. If we want to find the largest of the `double` values mapped to the integer keys in the range 1 to 100, we might be tempted to write the following loop:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
  largest = std::max(largest, theMap[i]);
```

If `theMap` is `const`, the loop will not compile. If any of the keys in the range `[1, 100]` are absent from the map, then `theMap[i]` will return a default-constructed `double` having value 0.0, which might or might not be desirable, depending on whether we want to treat missing values as truly having value 0.0 or whether they should be ignored (or,

equivalently, treated as having value -*INF*). Finally if, before executing this loop, `theMap` contains only, say, 5 entries, at the end of the loop it will contain at least 100 entries, most of which will have zero values.

There are alternatives that avoid all of these shortcomings, but the alternatives are significantly less elegant, and therefore more error prone. For example, the `at` member function looks a lot like `operator[]` and has none of the above shortcomings, but missing keys are handled by throwing exceptions, making it impractical for situations other than when the key is almost certain to exist. Moreover, a `try-catch` block within a loop is rarely a clean way to structure code:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
  try {
    largest = std::max(largest, theMap.at(i));
  } catch (const std::out_of_range&) { }
}
```

The above code would work with a `const` map, would ignore missing elements (rather than treating them as zeros), and would not populate the map with useless entries, but many would argue that the loop is inelegant, at best. In most C++ implementations, it would be extremely inefficient unless key misses are rare.

The other obvious alternative uses the `find` member function:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
  auto iter = theMap.find(i);
  if (iter != theMap.end())
    largest = std::max(largest, iter->second);
}
```

This version of the loop is only slightly more verbose than our starting point and avoids all of the issues, but the use of iterators, the need to call *two* member functions (`find` and `end`), and having to extract the `second` element of the element pair for what should be a *simple* operation increases the cognitive load on both the programmer and the reader. Moreover, a generic use of `find` can yield a subtle bug. Consider function template `f`:

```
template <class Key, class Value>
void f(const Key& k, const std::map<Key, Value>& aMap)
{
  Value obj = some-default-obj-value-expression;
  auto iter = aMap.find(k);
  if (iter != aMap.end())
    obj = iter->second;
  // code that uses `obj` ...
}
```

The above code will not compile unless `Value` is copy-assignable. Worse, unless tested with a non-assignable type, the bug could go undetected for a long time. One fix would be initialize `obj` in a single expression:

```
Value obj = aMap.count(k) ? aMap.at(k) : some-default-obj-value-
        expression;
```

While correct, this solution involves two lookup operations: One for the `count` and one for the `at`. A better fix requires a bit more code:

```
auto iter = aMap.find(k);
Value obj = iter != aMap.end() ? iter->second : some-default-obj-value-
        expression;
```

Note that the last solution again involves `iterator`, `pair`, and a conditional expression, which is a far cry from the simplicity of `operator[]`.

3 Proposed feature

3.1 Overview

What's desired is a simple member function that, given a key, returns the mapped value if the key exists and a specified *alternative* if the value does not exist. Inspired by a similar member of Python dictionaries, I propose a `get` member function and related `get_ref` and `get_as<T>` member functions for all C++ dictionary-like associative containers. A slightly simplified version of set of the proposed prototypes is:

```
// Return by value
template <class... Args>
  mapped_type get(const key_type& key, Args&&... args) const;

// Return by reference
template <class Arg>
   common_reference_t<mapped_type&,          Arg&> get_ref(const key_type&
        key, Arg& ref);

template <class Arg>
    common_reference_t<const mapped_type&, Arg&> get_ref(const key_type&
        key, Arg& ref) const;

// Return as a specific type
template <class R, class... Args>
  R get_as(const key_type& key, Args&&... args);

template <class R, class... Args>
  R get_as(const key_type& key, Args&&... args) const;
```

In each case, if `key` is found, the corresponding mapped value is returned, otherwise, an alternative return value is constructed from the remaining arguments. The differences among the proposed member functions are in the way that the return types are determined.

§ 3.2 The `get` member function

The proposed `get` member is the simplest to use and is suitable for types having relatively inexpensive copy constructors. If the key is found, the return value is a *copy* of the mapped value, otherwise the return value is constructed from `args`. If the key is not found and `args` is an empty pack, then the return value is default constructed.

Returning by value is not as expensive as it once was because the materialization rules (formerly *copy elision* rules) mean that fewer temporaries are created. Moreover, returning by value allows the alternative value to be specified as a prvalue such as the literal `-1` or `nullptr`.

§ 3.3 The `get_ref` member function

Sometimes returning a reference to the mapped item is desirable, either for efficiency (to avoid an expensive copy constructor) or functionality (if you want to modify the item or inspect its address). The `get_ref` member provides the necessary functionality. Consider an example where we modify mapped values through the reference returned from `get_ref`:

```
std::map<std::string, int> theMap;
// ...
// Increment the entries matching `names`, but only if they are already in
        `theMap`.
for (const auto& name : names) {
  int temp = 0;  // Value is irrelevant
  ++theMap.get_ref(name, temp);  // increment through reference
  // Possibly-modified value of `temp` is discarded here.
}
```

Note that the second argument to `get_ref` must be a single lvalue, in this case a local variable that will be discarded whether or not it is modified.

The function's interface is designed to avoid accidentally binding an rvalue to a `const` reference argument, even when the map is `const`. For example, the temporary object created when converting a string literal argument to `std::string` would go out of scope before `ref` goes out of scope (lifetime extension is not transitive). The interface is designed so that this error will be caught at compile time:

```
void f(const std::map<int, std::string>& theMap) {
  const std::string& ref = theMap.get_ref(0, "zero");  // ERROR: temporary
        `std::string("zero")`
  // ...
}
```

Finally, the return type for `get_ref` is the *common reference type* between `mapped_type&` (or `const mapped_type&`) and the reference passed as the second argument. This definition makes it easy to generate a meaningful result when mixing cv-qualifications or when mixing base-class references with derived-class references in a single call:

```
void g(const std::map<int, int>& theMap)
{
  // ...
  int alt = 0;
  auto& ref = theMap.get_ref(key, alt);  // `ref` has type `const int&`
  // ...
}
```

The result of `get_ref` is `const` if either or both of the map or alternative reference are `const`. Thus `ref` is `const` in the example above because `theMap` is a `const` lvalue, even though `alt` is non-const. A similar normalization occurs when mixing base-class and derived-class references:

```
std::map<int, Derived> theMap;
// ...
Base alt{ ... };
auto& ref = theMap.get_ref(key, alt);  // `ref` has type `Base&`
```

§ 3.4 The `get_as` member function

The `get_as` member allows the user to specify the desired return type. It is useful when the desired type is *convertible from* the mapped type and where a more efficient construction is possible for the alternative value. A ubiquitous example is `std::string_view`, which can be constructed efficiently from a `std::string` or a `char` array. The `get_as` member allows the `mapped_type` to be `std::string` and the alternative value to be `char[]` without generating extra temporary values:

```
std::map<int, std::string> theMap;
// ...
std::string_view sv = theMap.get_as<std::string_view>(key, "none");
```

The above example creates the resulting `std::string_view` from either the `std::string` stored in the map or the `const char[5]` passed as the alternative value, without creating an intervening `std::string`. It would be inefficient to convert the `char` array to `std::string` before converting it to a `std::string_view` and it would create lifetime issues:

```
//  BUG:  Dangling  reference  converting  returned  temporary  `string`  to
        `string_view`
std::string_view sv = theMap.get(key, "none");

// ERROR: cannot bind temporary `string` to `string&` parameter
std::string_view sv = theMap.get_ref(key, "none");
```

If the specified return type is a reference type, then `get_as` behaves much like `get_ref` except that the return type is exactly the specified type, rather than the deduced common reference type between the map and the alternative argument:

```
std::map<int, int> theMap;
const int zero = 0;
```

```
auto& v1 = theMap.get_as<int&>(0, zero);       // ERROR: `const` mismatch
auto& v2 = theMap.get_as<const int&>(0, zero); // OK
```

§ 4 Before/After Comparisons

The following tables shows how operations are simplified using the proposed new member functions. In these examples `K`, `T`, and `U` are object types; `m` is an object of (possibly *cv*-) `std::map<K, T>`, `unordered_map<K, T>` or `flat_map<K, T>`, `k` is a value compatible with `K`, `v` is an lvalue of type (possibly *cv*-) `T`, and `a1..aN` are arguments that can used to construct an object of type `T`.

Before	After/Alternative

`auto iter = m.find(k);`
`T x = iter == m.end() ? T{} :`
`  iter->second;`
	`T x = m.get(k);`

`auto iter = m.find(k);`
`T x = iter == m.end() ?`
`  T{a1...aN} : iter-`
`  >second;`
	`T x = m.get(k, a1...aN);`

`T v;`
`auto iter = m.find(k);`
`T& x = iter == m.end() ? v :`
`  iter->second;`
	`T v;`
	`T& x = m.get_ref(k, v);`

`std::map<K, std::vector<U>> m{`
`  ... };`
`auto iter = m.find(k);`
`std::span<U> x = iter ==`
`  m.end() ?`
`  std::span<U>{} :`
`  iter->second;`
	`std::map<K, std::vector<U>> m{ ...`
	`  };`
	`std::span<U> x =`
	`  m.get_as<std::span<U>>(k);`

`std::map<K, std::vector<U>> m{`
`  ... };`
`const std::array<U, N> preset{`
`  ... };`
`auto iter = m.find(k);`
`std::span<const U> x = iter ==`
`  m.end() ?`
`  std::span<const U>`
`  {preset} : iter-`
`  >second;`
	`std::map<K, std::vector<U>> m{ ...`
	`  };`
	`const std::array<U, N> preset{ ...`
	`  };`
	`std::span<const U> x =`
	`  m.get_as<std::span<const`
	`  U>>(k, preset);`

`std::unordered_map<K, U*> m{`
`  ... };`
`auto iter = m.find(k);`
	`std::unordered_map<K, U*> m{ ... };`
	`U* p = m.get(k, nullptr);`
	`if (p) {`

Before	After/Alternative

`if (iter != m.end()) {`
`  U* p = iter->second;`
`  // ...`
`}`
	`  // ...`
	`}`

`auto iter = m.find(k);`
`if (iter != m.end()) {`
`  T& r = iter->second;`
`  // ...`
`}`
	`T not_found;`
	`T& r = m.get_ref(k, not_found);`
	`if (&r != &not_found) {`
	`  // ...`
	`}`

5 Alternatives considered

5.1 Names

The name `get` is borrowed from the Python dictionary member of the same name. Other names considered were `lookup`, `lookup_or`, and `get_or`; the latter two follow the precedent of `std::optional<T>::value_or`. The `get` name was selected for conciseness and familiarity.

5.2 Return `optional` or `expected`

Some of the deficiencies of `operator[]` could be addressed by adding a member function that returns an `optional<mapped_type>` object:

```
// ALTERNATIVE (not proposed)
auto get(const key_type& k) const -> optional<mapped_type>;
```

Better yet, if [P2988R1] is adopted, it could return an `optional<T&>`:

```
// ALTERNATIVE (not proposed) with `optional<T&>` per [P2988R1]
auto get(const key_type& k)       -> optional<mapped_type&>;
auto get(const key_type& k) const -> optional<const mapped_type&>;
```

While this approach solves the issues listed in the motivation section of this paper, typical use would require an `if` statement (making it almost as verbose to use as `find`), or the use of `value_or`, making it syntactically similar to, but more complicated, than the proposed `get` member.

Returning `T` (proposed)	Returning `optional<T>` (not proposed)
`product *= theMap.get(k, 1);`	`product *= theMap.get(k).value_or(1);`

In the second column, a call to `get`, followed by a call to `value_or` on the returned `optional` causes the mapped value to be copied twice: first into the `optional` value, then again into the temporary returned by `value_or`. Even if `optional<T&>` is adopted, per

[P2988R1], there would be no equivalent to `get_ref` (because [P2988R1] currently has `value_or` return by value, even for a reference `value_type`) or `get_as`. Ultimately, it was decided that the `get` interface returning `optional` does not provide a substantial advantage over the proposed interface, especially in the absence of optional references.

§ 5.3 Combining `get` and `get_ref` into a single function

If the `Args` parameter pack happens to consist of a single lvalue reference that is compatible with `mapped_type&`, it would be possible to return a reference instead of a value, obviating `get_ref` as a separate function. Such a "simplification" was seen as confusing, especially to a human reader, who must analyze the code carefully to determine whether non-const operations on the returned entity might change a value in the original map.

§ 5.4 Using `get_as<mapped_type&>` instead of `get_ref`

During development of the reference implementation for this proposal, `get_as` was seen as sufficient for returning a reference to a member object. Trying to use the library in usage examples turned up some disadvantages, however:

— It was verbose, especially when `const` qualifications were added.

— It was tedious getting `const` qualifications correct when using non-`const` maps with `const` alternative values.

The introduction of `get_ref` made the code easier to read and write, which is the point of this entire paper.

§ 5.5 Rvalue overloads

I considered adding an rvalue-reference overload for `get_ref` that would accept rvalue reference alternative and return an rvalue reference. None of the other element lookup functions are overloaded in this way, however (e.g., `std::move(m)[k]` does not return an rvalue reference), so an rvalue overload would be novel and inconsistent. If, in the future, `operator[]` and `at` were to be enhanced with rvalue overloads, then `get_ref` should be, as well.

§ 5.6 Free functions instead of members

It is possible to provide the functionality described in this paper using namespace-scope functions, without modifying `std::map` and `std::unordered_map`:

```
template <class Map, class K, class... Args>
  typename Map::mapped_type get(Map& m, const K& k, Args&&... args);

auto x = get(m, k, a1...aN);
```

One benefit to this approach is that the `get` template can handle any dictionary type that follows the general formula of `std::map` (i.e., having a `mapped_type`, and a `find` that returns an `iterator` pointing to a key-value `pair`). Such an approach has disadvantages, however:

— It is less intuitive because it puts `get` outside of the map interface.

— The short names can cause ADL issues.

In the end, it seemed wise to stick to member functions.

§ 6 Implementation Experience

An implementation, with tests and usage examples, can be found at https://github.com/phalpern/WG21-halpern/tree/main/P3091-map_lookup.

§ 7 Wording

Changes are relative to the 2023-12-18 working draft, [N4971].

In 24.4.4.1 [map.overview][1]/2, insert the `get`, `get_ref`, and `get_as` element-access members:

```
// 24.4.4.3 [map.access], element access
mapped_type& operator[](const key_type& x);
mapped_type& operator[](key_type&& x);
template<class K> mapped_type& operator[](K&& x);
mapped_type&       at(const key_type& x);
const mapped_type& at(const key_type& x) const;
template<class K> mapped_type&       at(const K& x);
template<class K> const mapped_type& at(const K& x) const;
template <class... Args> mapped_type get(const key_type& x,
Args&&... args) const;
template <class K, class... Args> mapped_type get(const K& x,
Args&&... args) const;
template <class U>
    common_reference_t<mapped_type&,           U&> get_ref(const
key_type& x, U& ref);
template <class U>
     common_reference_t<const mapped_type&, U&> get_ref(const
key_type& x, U& ref) const;
template <class K, class U>
  common_reference_t<mapped_type&      , U&> get_ref(const K& x,
U& ref);
template <class K, class U>
  common_reference_t<const mapped_type&, U&> get_ref(const K& x,
```

```
U& ref) const;
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args);
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args) const;
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args);
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args) const;
```

Wording question: In my reference implementation, I declared all of these functions as `[[nodiscard]]`, but the standard currently seems to use that attribute very parsimoniously, mostly (but not exclusively) in situations where a resource might leak or where a function returning a result could be confused with a function that modifies its input. These functions fit neither criteria, should they be `[[nodiscard]]` or not?

At the end of 24.4.4.3 [map.access], add these descriptions:

```
template <class... Args> mapped_type get(const key_type& x,
Args&&... args) const;
template <class K, class... Args> mapped_type get(const K& x,
Args&&... args) const;
```

> *Constraints*: For the second overload, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.
>
> *Returns*: `get_as<mapped_type>(x, std::forward<Args>(args)...)`.

```
template <class U>
    common_reference_t<mapped_type&,           U&> get_ref(const
key_type& x, U& ref);
template <class U>
     common_reference_t<const mapped_type&, U&> get_ref(const
key_type& x, U& ref) const;
template <class K, class U>
  common_reference_t<mapped_type&      , U&> get_ref(const K& x,
U& ref);
template <class K, class U>
  common_reference_t<const mapped_type&, U&> get_ref(const K& x,
U& ref) const;
```

> *Constraints*: For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.
>
> *Mandates*:

— The expression `find(x)` is well-formed.

— The result type is a reference type.

— `find(x)->second` and `ref` are both convertible to the result type.

Preconditions: The expression `find(x)` has well-defined behavior.

Returns: `find(x)->second` if `find(x) == end()` is `false`, otherwise `ref`.

Throws: nothing

Complexity: Logarithmic

Editorial comment: The last two *mandate*s are expressed in English rather than code because to express them in code would involve a hard-to-read construction like `is_convertible_v<decltype((find(x)->second)), common_reference_t<decltype((find(x)->second)), U&>`. This could be simplified using some "Let *CV_MAPPED_TYPE* be…" language if LWG prefers it.

```
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args);
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args) const;
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args);
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args) const;
```

Constraints: For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

Mandates:

— The expression `find(x)` is well-formed.

— `find(x)->second` is convertible to `R`.

— `is_constructible<R, Args...>` is `true`.

— If `R` is a reference type, then `sizeof...(Args)` is `1` and `is_lvalue_reference_v<Args...>` is `true`.

Preconditions: The expression `find(x)` has well-defined behavior.

Returns: `find(x)->second` if `find(x) == end()` is `false`, otherwise `R(std::forward<Args>(args)...)`.

Throws: nothing unless the result object's constructor throws.

Complexity: Logarithmic

In 24.5.4.1 [unord.map.overview]/3, insert the `get`, `get_ref`, and `get_as` element-access members:

```
// 24.5.4.3 [unord.map.elem], element access
mapped_type& operator[](const key_type& x);
mapped_type& operator[](key_type&& x);
template<class K> mapped_type& operator[](K&& x);
mapped_type&       at(const key_type& x);
const mapped_type& at(const key_type& x) const;
template<class K> mapped_type&       at(const K& x);
template<class K> const mapped_type& at(const K& x) const;
template <class... Args> mapped_type get(const key_type& x,
Args&&... args) const;
template <class K, class... Args> mapped_type get(const K& x,
Args&&... args) const;
template <class U>
    common_reference_t<mapped_type&,           U&> get_ref(const
key_type& x, U& ref);
template <class U>
     common_reference_t<const mapped_type&, U&> get_ref(const
key_type& x, U& ref) const;
template <class K, class U>
  common_reference_t<mapped_type&      , U&> get_ref(const K& x,
U& ref);
template <class K, class U>
  common_reference_t<const mapped_type&, U&> get_ref(const K& x,
U& ref) const;
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args);
template <class R, class... Args> R get_as(const key_type& x,
Args&&... args) const;
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args);
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args) const;
```

At the end of 24.5.4.3 [unord.map.elem], add these descriptions:

```
template <class... Args> mapped_type get(const key_type& x,
Args&&... args) const;
template <class K, class... Args> mapped_type get(const K& x,
Args&&... args) const;
```

Constraints: For the second overload, the *qualified-id*s `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

Returns: `get_as<mapped_type>(x, std::forward<Args>(args)...)`.

```
template <class U>
    common_reference_t<mapped_type&, U&> get_ref(const key_type& x, U& ref);
template <class U>
     common_reference_t<const mapped_type&, U&> get_ref(const key_type& x, U& ref) const;
template <class K, class U>
  common_reference_t<mapped_type&, U&> get_ref(const K& x, U& ref);
template <class K, class U>
  common_reference_t<const mapped_type&, U&> get_ref(const K& x, U& ref) const;
```

Constraints: For the third and fourth overloads, the *qualified-id*s `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

Mandates:

— The expression `find(x)` is well-formed.

— The result type is a reference type.

— `find(x)->second` and `ref` are both convertible to the result type.

Preconditions: The expression `find(x)` has well-defined behavior.

Returns: `find(x)->second` if `find(x) == end()` is `false`, otherwise `ref`.

Throws: nothing

Complexity: Logarithmic

```
template <class R, class... Args> R get_as(const key_type& x, Args&&... args);
template <class R, class... Args> R get_as(const key_type& x, Args&&... args) const;
template <class R, class K, class... Args> R get_as(const K& x, Args&&... args);
```

```
template <class R, class K, class... Args> R get_as(const K& x,
Args&&... args) const;
```

Constraints: For the third and fourth overloads, the *qualified-id*s `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

Mandates:

— The expression `find(x)` is well-formed.

— `find(x)->second` is convertible to `R`.

— `is_constructible<R, Args...>` is `true`.

— If `R` is a reference type, then `sizeof...(Args)` is `1` and `is_lvalue_reference_v<Args...>` is `true`.

Preconditions: The expression `find(x)` has well-defined behavior.

Returns: `find(x)->second` if `find(x) == end()` is `false`, otherwise `R(std::forward<Args>(args)...)`.

Throws: nothing unless the result object's constructor throws.

Complexity: Logarithmic

§ 8 References

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++. https://wg21.link/n4971

[P0429R9] Zach Laine. 2022-06-17. A Standard flat_map. https://wg21.link/p0429r9

[P2767R2] Arthur O'Dwyer. 2023-12-09. flat_map/flat_set omnibus. https://wg21.link/p2767r2

[P2988R1] Steve Downey, Peter Sommerlad. 2024-01-05. std::optional<T&>. https://wg21.link/p2988r1

---

1. All citations to the Standard are to working draft N4971 unless otherwise specified.↩